



University of AJ&K, Neelum Campus

Department of Computer Science & IT

Computer Organization and Assembly Language

Assignment No. 3

Topic

Assembly Program to Print a Message

Submitted By:

Jawad Ahmed

4 Sept 2025

Course Information

Course Name: Computer Organization and Assembly Language

Course Code: CSC-542

Course Instructor: Abdual Qaddous

Student Information

Name: Jawad Ahmed Khawaja

Roll No: 04

Semester: *BSCS – 4th*

Submission Date: 4-September-2025

Question

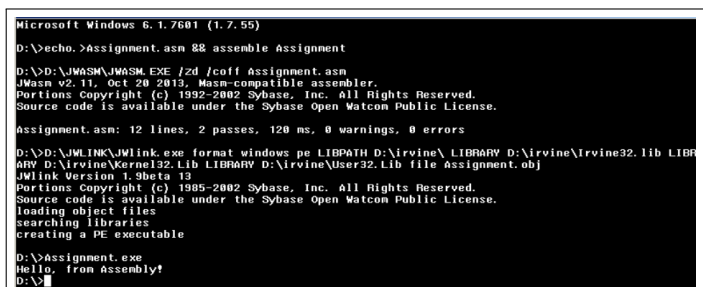
Write down a program in Assembly Language to print the message of your own choice.

Code

Assembly Code

```
.686
INCLUDE Irvine32.inc
.data
    msg BYTE "Hello, from Assembly!",0
.code
Main PROC
    lea edx, msg
    Call Clrscr
    Call WriteString
    EXIT
Main ENDP
END Main
```

Output



```
Microsoft Windows [6.1.7601 (1.7.55)]
D:\>echo.>Assignment.asm 88 assemble Assignment
D:\>D:\Jvasm\Jvasm.exe /zd /coff Assignment.asm
Jvasm v2.11, Oct 28 2015, Masm-compatible assembler,
Portions Copyright (c) 1992-2002 Sybase, Inc. All Rights Reserved.
Source code is available under the Sybase Open Watcom Public License.
Assignment.asm: 12 lines, 2 passes, 120 ms, 0 warnings, 0 errors
D:\>D:\Jvlink\Jvlink.exe format windows pe LIBPATH D:\Irvine\ LIBRARY D:\Irvine\Irvine32.lib LIBR
ARY D:\Irvine\Kernel32.lib LIBRARY D:\Irvine\User32.lib file Assignment.obj
Jvlink Version 1.9beta 13
Portions Copyright (c) 1985-2002 Sybase, Inc. All Rights Reserved.
Source code is available under the Sybase Open Watcom Public License.
loading object files
searching libraries
creating a PE executable
D:\>Assignment.exe
Hello, from Assembly!
D:\>
```

Explanations of Code

1. `.686`

This directive tells the assembler to generate code for the Intel 686 (Pentium Pro) processor or higher. It sets the instruction set architecture.

2. `INCLUDE Irvine32.inc`

This line includes the Irvine32 library, which provides useful procedures (like `WriteString`, `Clsrscr`, etc.) for easier input/output operations in Assembly language.

3. `.data`

This section is used to declare initialized data or constants. Here we define variables that will be used in the program.

4. `msg BYTE "Hello, from Assembly!",0`

This defines a string variable named `msg`. The string is terminated with a null byte (0), which signals the end of the string.

5. `.code`

This marks the beginning of the code section where the instructions (executable code) are written.

6. `Main PROC`

It defines the beginning of the procedure (function) named `Main`. This is the entry point of the program.

7. `lea edx, msg`

It loads the effective address of the variable `msg` into the register `EDX`. The Irvine32 library procedures expect the address of the string in `EDX`.

8. `Call Clrscr`

It calls the procedure `Clrscr`, which clears the console screen.

9. `Call WriteString`

It calls the procedure `WriteString`, which prints the string located at the address in `EDX`.

10. **EXIT**

It ends the program and returns control to the operating system.

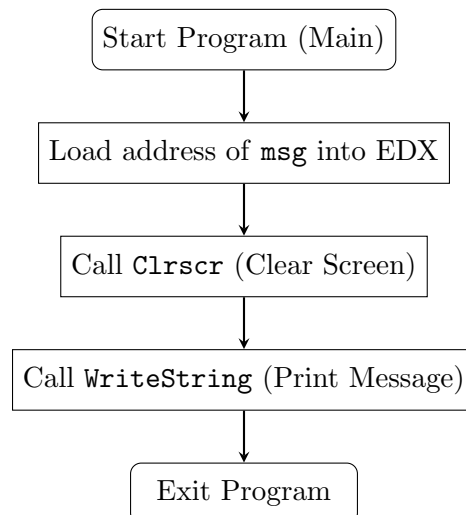
11. **Main ENDP**

It marks the end of the **Main** procedure.

12. **END Main**

It tells the assembler that the program execution should start at the label **Main**.

Flowchart of Program Execution



Summary

In this program, we learned how to display a simple message using Assembly Language. The program uses the `Irvine32.inc` library for input/output operations. This program demonstrates the basic structure of an Assembly Language program, including the use of data definition, procedure calls, and screen output.